

Accelerating Developer Velocity at Paystack.

Next-Gen API Infrastructure & Developer Experience

Enriched OpenAPI 3.0 Spec • Interactive Docs • Multi-Language SDK Generator • Modern CLI

Ecosystem Overview

- ▶ **1. Enriched OpenAPI Spec:** Single source of truth with quality metadata & 100% operationId coverage.
- ▶ **2. Interactive Docs:** Next.js & Markdoc interactive developer experience with live API execution.
- ▶ **3. Multi-Language SDK Gen:** 10-language SDK generator with exponential backoff & auto-idempotency.
- ▶ **4. Modern Scriptable CLI:** Fast, non-interactive terminal tool with automatic key resolution & zero-dependency webhooks.

```
# @title "Dev Ecosystem Health Check"

# @type "echo 'Checking Paystack Developer Tooling Suite...'"

echo 'Checking Paystack Developer Tooling Suite...'

# @wait 500ms

echo '[OK] paystack-spec-enriched (0 errors, 0 warnings)'

echo '[OK] paystack-docs (Next.js 14 + Markdoc)'

echo '[OK] paystack-sdk-gen (10 language targets)'

echo '[OK] paystack-cli (@paystack-oss/paystack-cli ~360ms latency)'
```

The "Developer Velocity" Lens

Time-to-Value (TTV). The core problem is not just missing features, we have friction that slows down the loop from "Idea" to "First Successful Transaction."

Ranked Gaps (Priority Order):

Critical: The "Static" Documentation & Manual Integration Loop

- ▶ The Gap: Docs are read-only. Developers must manually copy keys, switch tabs to the dashboard, configure Postman, and guess error states.
- ▶ Impact: High drop-off during activation. Senior developers (target market) view this as "legacy" compared to global standards set by payment giants like Stripe.
- ▶ Evidence: Paystack's own 2024 research noted developers want "better documentation to understand how to maximize use of tools" and cited a lack of advanced technical content.
- ▶ Why Fix First: It blocks everyone. You cannot activate a user who cannot successfully make their first API call within 5 minutes.

High: Unreliable & Fragmented Tooling (CLI & SDKs)

- ▶ The Gap: The CLI updates are far between (last update Aug 2024, relies on unmaintained libraries like vorpai, requires manual ngrok setup). SDKs are manually maintained, leading to likely hood of drift from the API spec.
- ▶ Impact: High churn for scaling companies. As merchants grow, they need scriptable CI/CD pipelines and type-safe code. The current state forces them to build internal wrappers or switch providers.
- ▶ Evidence: GitHub issues on PaystackOSS/paystack-cli show errors from 2020 that may still be unresolved. Community forks exist to fill the void.

Medium: Webhook Complexity

- ▶ The Gap: Testing webhooks requires third-party tools (ngrok) and manual configuration. There is no "one-click" local testing.
- ▶ Impact: Slows down development of critical post-payment flows (subscriptions, disputes).

The Enriched OpenAPI Spec

Single Source of Truth (`paystack-spec-enriched`)

Adopt an OpenAPI-First workflow where the Enriched Spec drives Docs, SDKs, and CLI simultaneously.

This serves as the single source of truth for the **Enriched Paystack OpenAPI Specification**. It synchronizes with the official `PaystackOSS/openapi` repository, applies structural, resilience, and developer-experience enrichments (`x-code-samples` , `x-operation-id` , `x-idempotency` , `x-retry-safe` , `x-dont-retry` , `x-pagination` , `x-deprecated-reason` , polymorphic webhook event schemas), and outputs unified distribution files (`dist/paystack-enriched.yaml` and `dist/paystack-enriched.json`).

- ▶ **How it helps Developers** : Eliminates drift. If the API changes, the docs, SDKs, and CLI update instantly. Guarantees type safety and accurate examples.
- ▶ **How it helps Paystack** : Reduces engineering maintenance overhead (no manual SDK updates). Increases activation rates by lowering TTV. Positions Paystack as "Enterprise Ready" for senior devs.

1. OpenAPI Spec Enrichment Features

Extension / Metadata	Purpose & Target Operations
<code>x-idempotency: true</code>	Added to resource creation & financial endpoints (<code>POST /transaction/initialize</code> , <code>POST /charge</code> , <code>POST /transfer</code> , <code>POST /refund</code> , <code>POST /customer</code> , etc.).
<code>x-retry-safe: true</code>	Added to 97 GET operations and idempotent POST operations.
<code>x-dont-retry: true</code>	Added to non-idempotent state-mutating actions (<code>POST</code> , <code>PUT</code> , <code>DELETE</code>).
<code>x-pagination</code>	Defined on 17 collection list endpoints (<code>/transaction</code> , <code>/customer</code> , <code>/transfer</code> , <code>/subscription</code> , <code>/plan</code> , <code>/refund</code> , <code>/subaccount</code> , <code>/dispute</code> , <code>/dedicated_account</code> , etc.).
<code>x-operation-id</code>	Standardized camelCase names with 100% coverage (163/163 HTTP operations mapped).
<code>x-code-samples</code>	Modular code snippets per endpoint (`snippets//[node`
<code>x-deprecated-reason</code>	Migration guidance notices for deprecated endpoints.
<code>WebhookEvent</code>	Polymorphic (<code>oneOf</code> + <code>discriminator</code>) typed event payload models (<code>ChargeSuccessEvent</code> , <code>TransferSuccessEvent</code> , <code>SubscriptionCreateEvent</code> , etc.).

Interactive API Documentation Portal

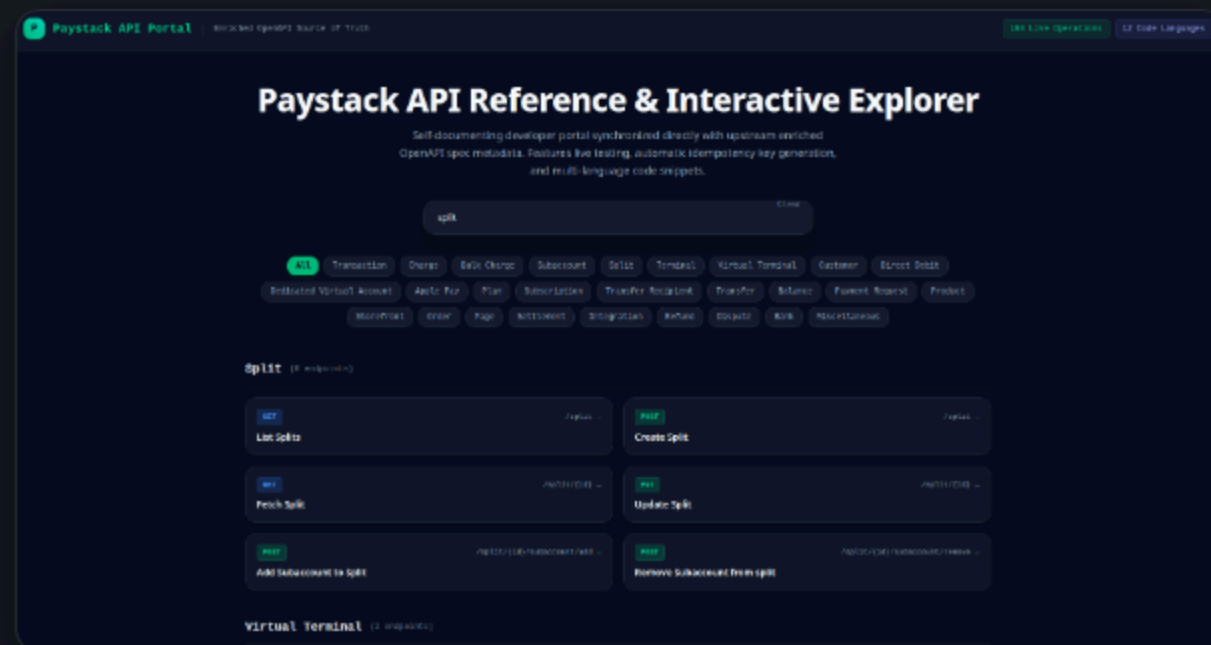
Next.js & Markdoc Interactive Portal (`paystack-docs`)

Interactive Explorer Feature Set

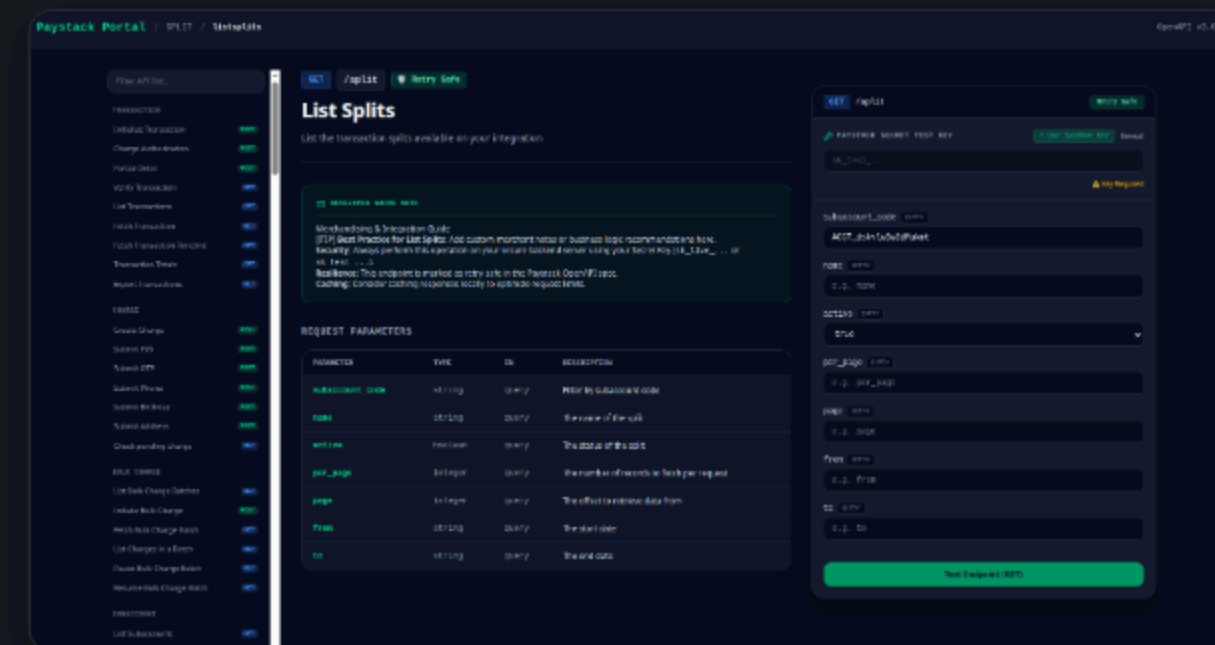
This is a 3-Column Developer Layout with the following features:

- ▶ **Sidebar Navigation:** Hierarchical category & operation tree generated directly from spec paths.
- ▶ **Markdoc Guide Overlays:** Markdown developer guides rendered with custom interactive tags.
- ▶ **Live API Explorer:** Right-hand interactive console for executing real HTTP requests.
- ▶ **Secure Key Proxy:** Encapsulates secret key injection without exposing raw credentials in browser memory.

Main Page



Split API Endpoint Interaction Page



Multi-Language Client SDK Pipeline

Enterprise Multi-Language SDK Generator (`paystack-sdk-gen`)

10 Target Languages & Decoupled Architecture

- ▶ **10 Language Matrix:** TypeScript, Python, Go, Java, PHP, C#, Ruby, Flutter, Android (Kotlin), and iOS (Swift). OpenAPI Generator supports almost 5 languages.
- ▶ **Decoupled Spec Storage:** Local spec copy (`spec/paystack-enriched.yaml`) enables fast, zero-network local generation scripts.
- ▶ **Automated Cross-Repo Sync:** Listens to `repository_dispatch` release events from `paystack-spec-enriched`. When a new release of `paystack-spec-enriched` occurs:
 - ▶ **GitHub Dispatch Trigger (`repository_dispatch`):**
 - ▶ `paystack-spec-enriched` emits a `repository_dispatch` event named `enriched-spec-updated`.
 - ▶ **Automated Spec Retrieval & Generation:**
 - ▶ GitHub Action step downloads `paystack-enriched.yaml` over HTTPS into `spec/paystack-enriched.yaml`.
 - ▶ Node.js generator engine reads `spec/paystack-enriched.yaml` and re-generates SDK packages.
 - ▶ Executes Prism Mock Server validation tests (`npx paystack-sdk-gen test`).
 - ▶ Auto-commits updated spec files and generated packages.

Local On-Demand Development Workflow

Internally, Paystack Developers can also run the generator CLI locally operating strictly on

`spec/paystack-enriched.yaml` for testing and validation purposes:

```
# 1. Verify local spec copy and trigger generation + Prism testing

npx paystack-sdk-gen sync


# 2. Generate specific target language SDKs from local spec copy

npx paystack-sdk-gen generate --lang typescript,python


# 3. Verify generated SDKs against local Prism Mock Server

npx paystack-sdk-gen test
```


Modern Scriptable CLI

Modern Paystack CLI (`@paystack-oss/paystack-cli`)

Improved DevEx

- ▶ **Scriptable Commander Architecture:** Replaces legacy interactive REPL with non-interactive, CI/CD-friendly subcommands.
- ▶ **Automatic Key Resolution:** Secure `paystack login` session storage; auto-resolves test/live keys per request.
- ▶ **40x Native Spec Speedup:** Offline bundled spec (`lib/paystack/openapi.json`) achieves ~360ms total command time.
- ▶ **JSON Output Piping:** Pass `--json` to pipe response payloads directly into `jq` or CI/CD pipelines.

```
# @title "CLI Verification"

# @type "paystack status"

paystack status

# @type "paystack api transaction initialize --email 'alex@example.com' --amount 50000 --domain test"

paystack api transaction initialize --email "alex@example.com" --amount 50000 --domain test
```

Paystack CLI modernization

- ▶ **Old REPL CLI:** Acted like a chat session (`paystack>`). This supports exploration, but breaks continuous integration and automation.
- ▶ **New Scriptable CLI:** Acts like standard Unix tools (e.g `git`, `docker`, `stripe-cli`). Every command can be scripted, automated, piped, and embedded in production build pipelines.

Why Scriptability Matters

Piping Data to Other Shell Tools (`jq`, `grep`, `curl`). The CLI supports the `--json` flag and non-interactive flags, you can chain commands together in bash:

```
# Fetch transaction status and extract just the status string using jq

STATUS=$(paystack api transaction verify --reference "qTPrJoy9Bx" --json | jq -r '.data.status')

if [ "$STATUS" = "success" ]; then

    echo "Payment confirmed!"

fi
```

Running in CI/CD Pipelines (e.g., GitHub Actions) to test API integrations automatically during pull requests:

```
# .github/workflows/test-integration.yml

steps:

  - name: Run Paystack API Healthcheck

    run: |

      paystack api transaction initialize \

        --email "ci-runner@example.com" \

        --amount 1000 \

        --domain test \

        --json > response.json
```

Zero-Dependency Local Webhooks & Auto-Updates

Zero-Dependency Webhook Engine & Package Updates

Webhook Simulation & Maintenance

- ▶ Authentic HMAC SHA-512: Computes authentic `x-paystack-signature` headers locally using secret keys.
- ▶ Zero Tunneling Overhead: `paystack webhook trigger` and `listen` forward events directly without `ngrok`.
- ▶ Self-Reporting Package Updates: 24-hour non-blocking background check alerts developers to npm updates.
- ▶ Single Update Command: `npm install -g @paystack-oss/paystack-cli` updates both binary logic and the bundled OpenAPI spec.

```
# @title "Webhook Trigger"

# @type "paystack webhook trigger charge.success --forward-to http://localhost:3000/api/webhook"

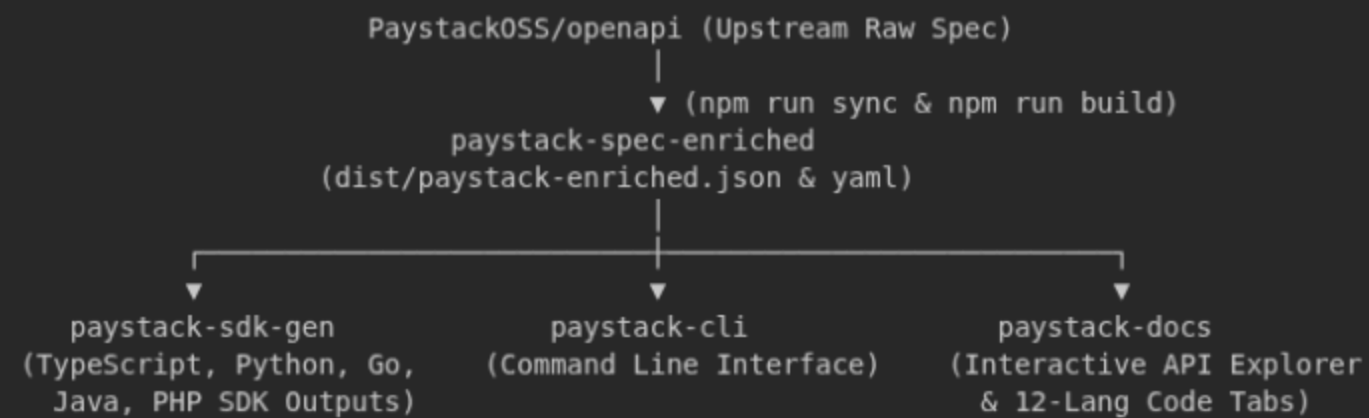
paystack webhook trigger charge.success --forward-to http://localhost:3000/api/webhook
```

Unified Developer Infrastructure

End-to-End Synergy Across the Ecosystem

Unified Architecture Impact

- ▶ **Single Source of Truth:** `paystack-spec-enriched` powers Docs, SDKs, and CLI in total sync.
- ▶ **Zero API Drift:** Automated GitHub Actions workflows ensure spec changes propagate instantly.
- ▶ **Global Standards:** Resilience, auto-pagination, typed errors, and high performance.
- ▶ **Developer First:** Built for seamless integration in local terminal sessions, IDEs, and CI/CD pipelines.



Ranked Gaps

- ▶ Static Docs (Activation Blocker).
- ▶ Fragmented Tooling (Retention Blocker).
- ▶ Webhook Friction (Complexity Blocker).

The Solution

- ▶ One Enriched OpenAPI First Spec → [Interactive Docs, Generated SDKs, Generated CLI].
- ▶ Interactive API Explorer with auto-injected keys, and real-time execution.
- ▶ Auto SDK Generation using OpenAPI Generator.
- ▶ Modern Scriptable CLI with Zero-Dependency Webhook Engine & Auto-Updates

Impact & ROI

- ▶ Activation: Reduce "Time to First Call" by ~70%.
- ▶ Retention: Signal to enterprise devs that Paystack is "modern."
- ▶ Efficiency: Engineering time saved on manual SDK updates.
- ▶ Benefit: Zero drift, automated maintenance.

Next Steps:

- ▶ Phase 1: Roll out Explorer to 10% of users.
- ▶ Phase 2: Deprecate manual SDKs in favor of generated ones.
- ▶ Phase 3: Rebuild CLI on top of the generated SDK.